

Practical session-6

Task 1 : Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
vennela_07@VennelaPC:~$ cd practical6
vennela_07@VennelaPC:~/practical6$ nano server.c
vennela_07@VennelaPC:~/practical6$ nano client.c
vennela_07@VennelaPC:~/practical6$ gcc server.c -o server
vennela_07@VennelaPC:~/practical6$ gcc client.c -o client
vennela_07@VennelaPC:~/practical6$ ./server
Server waiting for client...
Client: Hello
```

```
vennela_07@VennelaPC: ~/pr × + v
vennela_07@VennelaPC:~$ cd practical6
vennela_07@VennelaPC:~/practical6$ ./client
Waiting for server...
Enter message: Hello
Server: Server received: Hello
Enter message: |
```

```
vennela_07@VennelaPC: ~/pr × + v
GNU nano 7.2 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char buffer[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server waiting for client...\n");

    int fd1 = open(FIFO1, O_RDONLY);
    int fd2 = open(FIFO2, O_WRONLY);

    while (1) {
        read(fd1, buffer, sizeof(buffer));

        if (strcmp(buffer, "exit") == 0) {
            break;
        }
    }
}
```

```

    }

    printf("Client: %s\n", buffer);

    sprintf(buffer, "Server received: %s", buffer);
    write(fd2, buffer, strlen(buffer) + 1);
}

close(fd1);
close(fd2);

unlink(FIFO1);
unlink(FIFO2);

printf("Server terminated.\n");

return 0;
}

```

```

GNU nano 7.2 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char buffer[100];

    printf("Waiting for server...\n");

    int fd1 = open(FIFO1, O_WRONLY);
    int fd2 = open(FIFO2, O_RDONLY);

    while (1) {
        printf("Enter message: ");
        fgets(buffer, sizeof(buffer), stdin);

        buffer[strcspn(buffer, "\n")] = '\0';

        write(fd1, buffer, strlen(buffer) + 1);

        if (strcmp(buffer, "exit") == 0) {
            break;
        }

        read(fd2, buffer, sizeof(buffer));

        printf("Server: %s\n", buffer);
    }

    close(fd1);
    close(fd2);

    return 0;
}

```

Task 2 : Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
vennela_07@VennelaPC: ~/pr × + v
vennela_07@VennelaPC:~$ cd practical6
vennela_07@VennelaPC:~/practical6$ nano task2.c
vennela_07@VennelaPC:~/practical6$ gcc task2.c -o task2
vennela_07@VennelaPC:~/practical6$ ./task2
Process ID: 39650
Waiting for signals...

SIGUSR1 received
```

```
vennela_07@VennelaPC:~$ cd practical6
vennela_07@VennelaPC:~/practical6$ kill -SIGUSR1 1234
-bash: kill: (1234) - No such process
vennela_07@VennelaPC:~/practical6$ kill -SIGUSR1 5678
-bash: kill: (5678) - No such process
vennela_07@VennelaPC:~/practical6$ kill -SIGUSR1 39650
vennela_07@VennelaPC:~/practical6$
```

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {
    if (sig == SIGINT) {
        printf("\nSIGINT received\n");
    }
    else if (sig == SIGTERM) {
        printf("\nSIGTERM received\n");
    }
    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received\n");
    }
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```